

Split 13 — Casos prácticos docentes

Objetivo del split

El objetivo de este split ha sido crear el bloque inicial de **Docencia** dentro de AulaNomina, separándolo claramente del ERP laboral.

Hasta ahora AulaNomina funcionaba principalmente como simulador ERP/HCM: empresas, centros, trabajadores, contratos, incidencias, documentos y nóminas. Con este split se ha añadido una capa docente encima del ERP para permitir que un profesor pueda crear ejercicios, asignarlos a alumnos o grupos, registrar entregas, corregirlas y hacer seguimiento del avance.

El objetivo no era todavía crear una plataforma educativa completa con autenticación real, sino una demo funcional y visual que permita enseñar el flujo docente completo:

Profesor crea caso práctico

↓

Profesor asigna caso a grupo o alumno

↓

Alumno ve el caso en una vista simulada

↓

Alumno marca el caso como entregado

↓

Profesor registra/corriga entrega

↓

Sistema muestra progreso, alertas y vencimientos

Enfoque general

Se ha mantenido el principio de desarrollo del MVP:

Rapidez de validación

Demo visualmente profesional

Flujos docentes entendibles

Sin sobreingeniería

Sin autenticación real todavía

Sin integraciones externas reales

El módulo de docencia queda separado visualmente en el menú lateral mediante una línea negra divisoria. Esto permite distinguir claramente entre:

ERP laboral

Docencia / simulación educativa

1. Nueva sección Docencia en el menú lateral

Se ha creado una nueva sección independiente en el menú lateral:

DOCENCIA
PANEL PROFESOR
ALERTAS
CASOS PRÁCTICOS
ASIGNAR CASO
CORRECCIONES
VISTA ALUMNO
ALUMNOS
GRUPOS
PROGRESO

La sección se ha movido debajo de **Seg. Social** y se ha añadido una línea negra separatoria para diferenciar el ERP laboral de la parte docente.

Además, se ha corregido la navegación interna para que, cuando se accede desde botones internos como “Corregir entregas”, “Asignar caso” o “Ir”, el menú lateral marque correctamente el módulo activo.

2. Casos prácticos

Objetivo

Crear ejercicios docentes que simulen procesos laborales reales dentro del ERP.

Funcionalidad implementada

Se ha creado el módulo **Casos prácticos**, orientado a definir la estructura del ejercicio.

Permite:

Crear caso práctico
Definir título
Definir descripción
Definir dificultad
Definir estado
Añadir tareas internas
Eliminar tareas
Cargar casos demo

Estados del caso

Borrador
Activo
Archivado

Dificultades

Básico
Medio
Avanzado

Tareas del caso

Cada caso puede tener varias tareas:

Crear trabajador
Asignar empresa y centro
Crear contrato
Generar documentación
Preparar nómina
Registrar incidencia

Campos de cada tarea:

Título
Descripción
Módulo relacionado
Resultado esperado
Orden
Obligatoria / no obligatoria

Decisión importante

Se ha limpiado la pantalla de Casos prácticos para que no mezcle seguimiento y edición.

Ahora:

Casos prácticos = diseño del ejercicio
Progreso = seguimiento del avance
Correcciones = evaluación

Esto deja el flujo más claro y escalable.

3. Alumnos

Objetivo

Crear una base de alumnos dentro del módulo docente.

Funcionalidad implementada

Se ha creado el módulo **Alumnos**.

Permite:

- Crear alumno
- Editar alumno
- Desactivar alumno
- Listar alumnos
- Filtrar alumnos
- Cargar alumnos demo
- Asignar alumno a un grupo real

Campos principales

- Código de alumno
- Nombre
- Apellidos
- Email
- Grupo
- Centro educativo
- Estado
- Notas
- Activo / inactivo

Estados

- Activo
- Inactivo
- Finalizado

Mejora aplicada

Inicialmente el grupo del alumno era un campo de texto. Se ha mejorado para que exista una relación real:

Student → StudentGroup

Ahora el alumno tiene:

- group_id
- group_name
- group_display_name

Se mantiene group_name por compatibilidad, pero la relación real ya se hace mediante group_id.

4. Grupos

Objetivo

Crear grupos docentes para organizar alumnos por curso, centro o programa.

Funcionalidad implementada

Se ha creado el módulo **Grupos**.

Permite:

- Crear grupo
- Editar grupo
- Desactivar grupo
- Listar grupos
- Filtrar grupos
- Cargar grupos demo
- Ver número de alumnos asignados

Campos principales

- Código de grupo
- Nombre del grupo
- Curso académico
- Centro educativo
- Docente responsable
- Estado
- Notas
- Activo / inactivo

Estados

- Activo
- Inactivo
- Finalizado

Mejora aplicada

Los grupos muestran un contador real de alumnos asignados:

`student_count`

Esto permite ver rápidamente qué grupos tienen alumnos y cuáles están vacíos.

5. Asignar caso

Objetivo

Crear el puente entre los casos prácticos y los alumnos/grupos.

Antes existían casos, alumnos y grupos, pero faltaba la relación docente principal:

Este caso se asigna a este grupo o a este alumno.

Funcionalidad implementada

Se ha creado el módulo **Asignar caso**.

Permite:

Seleccionar caso práctico
Asignarlo a un grupo
Asignarlo a un alumno individual
Definir fecha límite
Definir profesor responsable
Añadir notas
Cambiar estado
Editar asignación
Eliminar asignación
Filtrar asignaciones
Cargar demo

Modelo creado

CaseAssignment

Campos principales:

id
case_study_id
student_id
group_id
assigned_by
assigned_at
due_date
status
notes
created_at

Validaciones

La asignación debe cumplir:

Debe tener un caso práctico
Debe asignarse a un alumno o a un grupo
No puede asignarse a alumno y grupo a la vez
El caso debe existir
El alumno debe existir si se asigna a alumno
El grupo debe existir si se asigna a grupo

Estados de asignación

assigned
in_progress
submitted
reviewed
approved
needs_revision

En la interfaz se muestran como:

Asignado
En curso

Entregado
Corregido
Aprobado
Requiere revisión

Resultado funcional

El flujo queda así:

Caso práctico → Asignación → Alumno/Grupo

Este era un punto clave para que el módulo docente dejase de ser una colección de pantallas sueltas y empezara a funcionar como plataforma educativa.

6. Correcciones vinculadas a asignaciones

Objetivo

Cambiar Correcciones para que dependan de una asignación y no de textos escritos manualmente.

Antes Correcciones funcionaba con campos como:

student_name
student_group
case_study_id

Esto era útil para una primera demo, pero poco escalable.

Ahora Correcciones se apoya en:

assignment_id

Funcionalidad implementada

La pantalla **Correcciones** permite:

Registrar entrega desde una asignación existente
Seleccionar asignación
Asignar estado inicial
Añadir nota inicial
Añadir feedback inicial
Corregir entrega
Actualizar nota
Actualizar feedback
Cambiar estado de corrección
Eliminar corrección

Modelo actualizado

Correction

Campos principales:

id

assignment_id
case_study_id
student_name
student_group
status
grade
teacher_feedback
reviewed_by
submitted_at
reviewed_at
created_at

Compatibilidad

Se mantienen temporalmente:

case_study_id
student_name
student_group

Esto evita romper datos antiguos o demo previa.

Pero el flujo nuevo recomendado es:

Assignment → Correction

Estados de corrección

pending_review
reviewed
approved
needs_revision

En la interfaz:

Pendiente de corregir
Corregido
Aprobado
Requiere revisión

Sincronización automática

Cuando se actualiza una corrección, también se actualiza el estado de la asignación:

pending_review → submitted
reviewed → reviewed
approved → approved
needs_revision → needs_revision

Resultado funcional

La cadena docente queda así:

Caso práctico

↓
Asignación
↓
Entrega / Corrección
↓
Nota y feedback

7. Progreso

Objetivo

Crear una vista específica para seguimiento del avance.

Funcionalidad implementada

Se ha creado el módulo **Progreso**.

Permite consultar:

Progreso global
Total de casos
Correcciones pendientes
Alumnos y grupos
Avance por caso
Tareas completadas
Tareas pendientes
Correcciones vinculadas

Cambio importante

Se movió a esta pantalla lo que antes estaba dentro de Casos prácticos:

Porcentaje de avance
Estado del caso
Tareas completadas
Seguimiento de tareas

Esto mejora la claridad:

Casos prácticos = crear estructura
Progreso = consultar avance

8. Panel del profesor

Objetivo

Crear una pantalla inicial para el profesor donde vea lo importante de un vistazo.

Funcionalidad implementada

Se ha creado el módulo **Panel profesor**.

Incluye:

- Casos activos
- Correcciones pendientes
- Casos que vencen esta semana
- Progreso medio
- Entregas registradas
- Casos fuera de plazo
- Alumnos / grupos
- Correcciones prioritarias
- Vencimientos próximos
- Casos recientes

Accesos rápidos

Desde el panel se puede ir directamente a:

- Asignar caso
- Corregir entregas
- Crear caso
- Ver progreso
- Ver asignaciones

Mejora de navegación

Se ha corregido la sincronización del menú lateral para que, al usar estos botones internos, el menú marque correctamente la pantalla destino.

Ejemplo:

- Panel profesor → Corregir entregas → se marca Correcciones
- Panel profesor → Asignar caso → se marca Asignar caso
- Alertas → Ir → se marca el módulo correspondiente

9. Vista alumno demo

Objetivo

Simular cómo vería el alumno sus ejercicios sin crear todavía autenticación real.

Funcionalidad implementada

Se ha creado el módulo **Vista alumno**.

Permite:

- Seleccionar alumno demo
- Ver sus casos asignados
- Ver estado de cada caso
- Ver instrucciones
- Ver tareas del caso
- Marcar caso como en curso
- Marcar caso como entregado
- Ver feedback del profesor
- Ver nota si existe corrección

Estados visibles para el alumno

Pendiente
En curso
Entregado
Corregido
Aprobado
Requiere revisión

Lógica de visibilidad

El alumno ve:

Asignaciones individuales hechas a él
Asignaciones hechas a su grupo

Esto permite simular correctamente:

Caso asignado a grupo completo
Caso asignado a alumno concreto

Resultado funcional

Ya se puede enseñar una demo completa:

Profesor crea caso
Profesor asigna caso
Alumno entra en vista alumno
Alumno ve instrucciones
Alumno marca como entregado
Profesor corrige
Alumno ve feedback

10. Alertas docentes

Objetivo

Crear una pantalla de vencimientos y avisos, alineada con la lógica laboral de plazos.

Funcionalidad implementada

Se ha creado el módulo **Alertas**.

Incluye alertas de:

Caso pendiente de entregar
Entrega fuera de plazo
Entrega próxima
Corrección pendiente
Caso sin tareas
Alumno sin grupo
Documento pendiente dentro de un caso

Clasificación de alertas

Las alertas se clasifican por severidad:

Críticas
Medias
Bajas

Filtros disponibles

Todas
Críticas
Medias
Bajas
Caso pendiente
Fuera de plazo
Corrección pendiente
Caso sin tareas
Alumno sin grupo
Documento pendiente

Acciones rápidas

Cada alerta tiene botón **Ir** para saltar al módulo relacionado:

Asignar caso
Correcciones
Casos prácticos
Alumnos
Progreso

Sentido funcional

Este módulo encaja especialmente bien con AulaNomina porque en gestión laboral y administrativa los plazos son fundamentales.

La misma filosofía se puede ampliar más adelante a:

Contratos próximos a vencer
Documentos caducados
IT pendiente de revisar
Nómina pendiente de preparar
Regularización pendiente
Fichero simulado pendiente de enviar

11. Navegación docente

Problema detectado

Al navegar desde botones internos, por ejemplo desde **Panel profesor** a **Correcciones**, la pantalla cambiaba correctamente, pero el menú lateral seguía marcando **Panel profesor**.

Solución aplicada

Se ha actualizado Sidebar .jsx para escuchar:

hashchange
aulanomina-route-change

Ahora el menú se sincroniza con el hash actual.

Resultado

La navegación interna queda coherente:

#teacher-dashboard → Panel profesor
#teaching-alerts → Alertas
#case-studies → Casos prácticos
#assignments → Asignar caso
#corrections → Correcciones
#student-demo → Vista alumno
#students → Alumnos
#groups → Grupos
#progress → Progreso

12. Backend creado o modificado

Modelos nuevos

Student
StudentGroup
CaseAssignment

Modelos modificados

Correction
CaseStudy
CaseTask

Schemas nuevos o modificados

StudentCreate
StudentUpdate
StudentResponse
StudentGroupCreate
StudentGroupUpdate
StudentGroupResponse
CaseAssignmentCreate
CaseAssignmentUpdate
CaseAssignmentResponse
CorrectionCreate
CorrectionUpdate
CorrectionResponse

CRUD nuevo o modificado

crud/student.py
crud/student_group.py
crud/case_assignment.py
crud/correction.py
crud/case_study.py

Rutas añadidas

GET /students
GET /students/next-code
POST /students
PUT /students/{id}
DELETE /students/{id}
POST /students/seed-demo
GET /student-groups
GET /student-groups/next-code
POST /student-groups
PUT /student-groups/{id}
DELETE /student-groups/{id}
POST /student-groups/seed-demo
GET /case-assignments
POST /case-assignments
PUT /case-assignments/{id}
DELETE /case-assignments/{id}
POST /case-assignments/seed-demo
GET /corrections
POST /corrections
PUT /corrections/{id}
DELETE /corrections/{id}
POST /corrections/seed-demo

13. Frontend creado o modificado

Nuevas páginas

CaseStudiesPage.jsx
CaseAssignmentsPage.jsx
CorrectionsPage.jsx
StudentsPage.jsx
StudentGroupsPage.jsx
ProgressPage.jsx
TeacherDashboardPage.jsx
StudentDemoViewPage.jsx
TeachingAlertsPage.jsx

Nuevos servicios frontend

caseStudyApi.js
caseAssignmentApi.js
correctionApi.js
studentApi.js

studentGroupApi.js

Componentes modificados

Sidebar.jsx

CaseStudiesRoute.jsx

14. Datos demo

Se han añadido datos demo para:

Casos prácticos

Tareas de casos

Alumnos

Grupos

Asignaciones

Correcciones

También se ha integrado en:

Arranque del backend

/demo/reset

El reset demo ahora repone también el bloque docente.

15. Resultado final del split

Al finalizar este split, AulaNomina ya tiene un bloque docente funcional con flujo completo:

Crear caso práctico

Crear grupo

Crear alumno

Asignar caso a grupo o alumno

Simular vista alumno

Marcar caso como entregado

Registrar corrección

Añadir nota y feedback

Consultar progreso

Ver alertas y vencimientos

Usar panel del profesor

Esto convierte AulaNomina en algo más que un ERP laboral simulado. Ahora empieza a funcionar como una plataforma educativa de simulación laboral.

16. Valor para la demo comercial

Este split aporta mucho valor comercial porque permite explicar AulaNomina como:

Un laboratorio docente de gestión laboral

Un ERP educativo para practicar procesos reales

Una plataforma para que el profesor cree y asigne casos

Un sistema donde el alumno practica dentro de un entorno tipo ERP

Un panel donde el profesor ve avances, entregas y correcciones

Para una universidad, FP o academia, el flujo ya es entendible:

Profesor: crea el ejercicio

Alumno: lo realiza en el entorno simulado

Profesor: corrige y da feedback

Centro: puede ver trazabilidad y progreso

17. Estado actual

El split queda finalizado con una base sólida para la parte docente.

Pendientes lógicos para futuros splits:

Autenticación real por roles

Vista alumno real con login

Asignaciones masivas más refinadas

Relación directa entre casos y acciones reales del ERP

Corrección automática parcial

Sistema de rúbricas

Exportar resultados

Panel de centro educativo

Alertas globales en barra superior

18. Conclusión

El Split 13 deja creada la primera versión funcional del sistema docente de AulaNomina.

Se ha conseguido separar claramente:

ERP laboral

Docencia

Alumno

Profesor

Progreso

Corrección

Alertas

El resultado es una demo mucho más potente, porque ya permite enseñar no solo gestión laboral simulada, sino también una metodología completa de aprendizaje práctico.